CampusTrade: Comprehensive Software Engineering Project Plan and Technical Architecture

Executive Summary and Strategic Vision

The modern university campus represents a dynamic, self-contained economic ecosystem characterized by transient populations and high-frequency, localized transactions. Within this environment, students constantly require the exchange of specific goods—ranging from textbooks and laboratory equipment to furniture, electronics, and dormitory essentials. However, a significant gap persists in the digital facilitation of these exchanges. Existing generic e-commerce platforms and social media groups are often clunky, impersonal, and fraught with security risks, leading to a phenomenon identified as the "Hyperlocal Trust Deficit". Students face challenges in verifying the legitimacy of buyers and sellers, coordinating safe meetups, and navigating platforms that are not tailored to the unique rhythms of academic life.

To resolve this inefficiency, the development of CampusTrade is proposed. CampusTrade is a highly specialized, peer-to-peer (P2P) student marketplace web application engineered to operate as a zero-commission, closed-ecosystem coordination engine between localized supply and demand. This project is specifically designed as a streamlined "mini project," adhering to the architectural constraint of avoiding complex external integrations. Instead of relying on convoluted third-party payment gateways, heavy message brokers, or external enterprise identity providers, the platform leverages self-contained, native technologies to achieve its goals. Furthermore, the platform integrates an "Ask Mode," shifting the traditional supply-side marketplace into a demand-driven ecosystem where students can post specific requests for items they need, accelerating the time-to-transaction.

This document outlines the exhaustive technical architecture, database design, backend infrastructure, and deployment strategy required to build, scale, and maintain CampusTrade. The architectural blueprint relies on a modern, decoupled technology stack featuring a frontend web application, a FastAPI backend for asynchronous API orchestration, and Supabase for PostgreSQL database management, authentication, and media storage. The deployment strategy utilizes Vercel's serverless infrastructure to ensure global scalability, low-latency performance, and continuous integration, completely bypassing traditional, complex server management. By adapting proven project management frameworks and emphasizing declarative infrastructure, this document serves as the foundational roadmap for the engineering and execution of the CampusTrade platform.

Project Identification and Structural Framework

To ensure systematic development, the organizational structure of the CampusTrade project is adapted from established academic and industrial software engineering frameworks, specifically mirroring the modular approach of complex analytical and recognition systems. In large-scale application development, segregating responsibilities into distinct functional teams prevents architectural bottlenecks and ensures that data pipelines remain clean and secure. By defining

these boundaries early, even a streamlined mini project can maintain the rigor of a production-grade enterprise application.
The foundational framework for CampusTrade requires the delineation of specific modules, each tasked with a core responsibility in the platform's lifecycle. This structural layout translates directly to the engineering workflow, ensuring that each component is developed in isolation before integration.

Module Designation	Core Responsibility	Technical Output
Platform Infrastructure	Designing the robust relational data models and schemas that power the entire system.	PostgreSQL tables, Foreign Key constraints, Row Level Security (RLS) policies.
Information Handling	Managing the secure ingestion of user metadata, including names and institutional emails.	Authentication workflows, user profile synchronization, data encryption protocols.
Content Acquisition	Governing the pre-processing of user-generated content, including listings and "Asks".	Image resizing, format conversion (WebP), payload validation, and content moderation.
Search and Discovery	Indexing structured catalog data to allow users to execute complex queries.	RESTful API endpoints for filtering by category, price, and request type.
Quality Assurance	Establishing continuous verification protocols to monitor system accuracy and security.	Unit testing, endpoint validation, and unauthorized access rejection protocols.
Documentation	Ensuring the backend infrastructure is comprehensively documented for frontend consumption.	Auto-generated OpenAPI specifications (Swagger/ReDoc).

By establishing these defined responsibilities early in the development lifecycle, the engineering team guarantees that the underlying infrastructure is highly cohesive, loosely coupled, and prepared for future scalability, fulfilling the requirements of a sophisticated yet tightly scoped software engineering endeavor.

Systems Architecture and Technology Stack Justification

The selection of the technology stack is the most critical decision in marketplace application development, directly impacting the platform's scalability, security, and long-term maintainability. Given the explicit requirement to design a web application without complex integrations, CampusTrade employs a decoupled, multi-tier architecture that minimizes reliance on disparate external services while maximizing built-in functionalities.

Backend Framework: FastAPI (Python)

FastAPI has rapidly emerged as the de facto standard for building high-performance RESTful APIs in Python, making it the ideal engine for this marketplace. Its architecture is built upon Starlette for asynchronous web routing and Pydantic for robust data validation, allowing it to

achieve throughput speeds comparable to natively asynchronous environments like Node.js and Go. For a marketplace handling concurrent requests—such as simultaneous item searches, active "Ask" polling, and image uploads—FastAPI's native support for asynchronous operations ensures that I/O-bound tasks do not block the main execution thread.

A critical advantage of FastAPI for a streamlined mini project is its automatic generation of interactive API documentation based on OpenAPI standards. As backend developers define routes and Pydantic models, FastAPI simultaneously constructs a Swagger UI and ReDoc interface. This self-documenting nature eliminates the need for complex, manual API contract maintenance, accelerating the integration process between backend logic and frontend client interfaces.

Database, Authentication, and Storage: Supabase

To fulfill the mandate of avoiding complex, piecemeal integrations, CampusTrade leverages Supabase as its comprehensive backend-as-a-service (BaaS). Rather than integrating a standalone database (like AWS RDS), a separate authentication provider (like Auth0), and a distinct storage bucket (like AWS S3), Supabase consolidates these requirements into a single, unified PostgreSQL ecosystem.

PostgreSQL's relational model is strictly ACID-compliant (Atomicity, Consistency, Isolation, Durability), ensuring that transactional data within the marketplace is processed reliably without corruption. Supabase extends PostgreSQL by seamlessly layering authentication directly into the database schema, allowing for complex Row Level Security (RLS) policies that tie data access directly to cryptographic user tokens. This consolidation radically simplifies the application architecture, permitting the development team to focus on core business logic rather than infrastructural plumbing.

Deployment Infrastructure: Vercel Serverless Functions

To ensure high availability and minimize infrastructure overhead, the FastAPI application will be deployed utilizing Vercel's serverless architecture. Traditional application deployment requires configuring virtual machines, managing reverse proxies (like Nginx), and handling operating system updates—tasks that introduce unnecessary complexity to a mini project. Vercel abstracts this entirely, allowing Python applications to run as serverless functions.

This deployment model automatically scales compute resources from zero to thousands of concurrent executions in response to traffic spikes, incurring compute costs only when API endpoints are actively processing requests. Furthermore, Vercel natively integrates with Git-based repositories, providing a continuous integration and continuous deployment (CI/CD) pipeline out of the box.

Database Design and Declarative Schemas

A marketplace is fundamentally a coordination engine, and its operational efficiency relies entirely on the structural integrity of its relational database. In an educational marketplace, data is inherently relational: students create profiles, profiles generate supply listings or demand "asks", these entities belong to categorical hierarchies, and users engage in transactional messaging regarding these entities.

Declarative Schema Management

Traditional database management relies on imperative migrations, where developers write sequential instructions to alter the database state. As complexity grows, this approach can fragment the understanding of the database structure across dozens of files, making it difficult to maintain. CampusTrade utilizes Supabase's Declarative Schemas, which allow engineers to define the final, desired state of the database in centralized files.
Under this paradigm, the schema files are version-controlled alongside the application codebase. The Supabase CLI automatically calculates the differential between the declared schema and the current database state, generating accurate migration files automatically. This "single pane of glass" approach reduces duplication, prevents schema drift, and vastly simplifies peer code reviews, ensuring the database evolves safely alongside the FastAPI backend.

Core Relational Entities and "Ask Mode" Integration

The PostgreSQL database for CampusTrade is constructed around highly normalized tables to eliminate data redundancy and enforce referential integrity via Foreign Keys. To satisfy the specific requirement of an "Ask Mode," the schema diverges from traditional supply-only marketplaces by treating buyer requests as primary, searchable entities alongside standard product listings.

Table Identifier	Primary Responsibility	Key Columns and Data Types	Structural Relationships and Constraints
auth.users	Supabase-managed authentication table storing core cryptographic credentials.	id (UUID), email (VARCHAR), encrypted_password (VARCHAR), created_at (TIMESTAMPTZ).	Managed internally by Supabase Auth; cannot be modified directly by the API to ensure cryptographic security.
public.profiles	Stores public-facing student metadata and academic verification status.	id (UUID), full_name (TEXT), university_domain (VARCHAR), is_verified (BOOLEAN).	id references auth.users(id) ON DELETE CASCADE. Serves as the primary actor entity for all marketplace actions.
public.categories	Hierarchical taxonomy for organizing marketplace items (e.g., Textbooks, Electronics).	category_id (SERIAL), name (VARCHAR), parent_id (INTEGER), slug (VARCHAR).	parent_id references categories(category_id). Facilitates infinitely nested subcategories for granular searching.
public.listings	Contains the physical items actively offered for sale, rent, or trade by students.	listing_id (UUID), seller_id (UUID), category_id (INTEGER), title (VARCHAR), description (TEXT), price (NUMERIC).	seller_id references profiles(id). category_id references categories(category_id).
public.asks	Powers the "Ask Mode"	ask_id (UUID),	requester_id references

Table Identifier	Primary Responsibility	Key Columns and Data Types	Structural Relationships and Constraints
	by allowing students to broadcast requests for specific items they need.	requester_id (UUID), category_id (INTEGER), title (VARCHAR), budget_max (NUMERIC), urgency (ENUM).	profiles(id). Flips the marketplace dynamic to demand-driven fulfillment.
public.media	Manages references to optimized images stored in Supabase Storage buckets.	media_id (UUID), entity_id (UUID), entity_type (ENUM: 'listing', 'ask'), storage_path (VARCHAR).	Polymorphic relationship allowing both listings and asks to attach visual media.

Row Level Security (RLS) and Automated Triggers

Because the PostgreSQL database interfaces directly with the backend API, strict access controls are mandatory at the database tier. PostgreSQL's Row Level Security (RLS) policies will be enforced on all public tables, creating an impenetrable layer of authorization. For example, an RLS policy on the listings and asks tables will dictate that UPDATE or DEL[span_48](start_span)[span_48](end_span)[span_50](start_span)[span_50](end_span)ETE operations can only be executed where the execution context matches the owner's UUID (auth.uid() = seller_id). This ensures that malicious actors cannot modify or delete items belonging to other students, even if they bypass the application layer.
Furthermore, to maintain seamless synchronization between the immutable, Supabase-managed aut[span_20](start_span)[span_20](end_span)h.users table and the mutable public.profiles table, a PostgreSQL trigger function will be implemented. This function, typically defined as handle_new_user(), listens for an AFTER INSERT event on the authentication table. Upon detecting a new registration, it automatically generates a corresponding row in the profiles table, safely extracting metadata such as the student's name from the raw JSON payload provided during the initial signup phase. This eliminates the need for dual-write logic in the FastAPI application, ensuring data consistency at the lowest level of the stack.

Backend API Engineering and Domain-Driven Structure

A critical factor in the long-term success and maintainability of a FastAPI project is its directory organization. For a multi-faceted marketplace handling authentication, listings, asks, and media, placing all routing and logic in a centralized script creates insurmountable technical debt and merge conflicts. CampusTrade adopts a Domain-Driven Design (DDD) folder structure, separating concerns across distinct architectural layers: routing, business logic, data validation, and database persistence.

Strategic Directory Organization

The repository is structured to ensure that incoming HTTP requests are handled by thin controllers, while heavy processing is delegated to dedicated service layers.

Directory / Module Path	Architectural Responsibility	Component Examples
app/main.py	The ASGI application entry point. Initializes the FastAPI instance, configures CORS middleware, and mounts API routers.	app = FastAPI(), app.include_router()
app/api/routers/	Contains the HTTP route definitions. Acts as a thin controller layer prioritizing request intake and response dispatching without heavy business logic.	users.py, listings.py, asks.py, auth.py
app/services/	Houses the core business logic, orchestration, and complex data manipulation. Fully testable independent of HTTP context.	listing_service.py, verification_service.py
app/repositories/	Manages all direct database interactions using the official Supabase Python client. Isolates CRUD operations from business rules.	user_repository.py, item_repository.py
app/schemas/	Defines Pydantic models utilized for strict request payload validation and safe response serialization.	ListingCreate, AskResponse, UserPayload
app/core/	Manages cross-cutting system concerns, including environment variable injection, dependency management, and logging configurations.	config.py, security.py, dependencies.py

RESTful Endpoint Design and Rigorous Data Validation

The API is engineered to strictly adhere to RESTful conventions, utilizing standard HTTP verbs to manipulate resources predictably. Endpoint paths are pluralized and logically nested; for instance, retrieving all "Asks" posted by a specific user is routed through GET /users/{user_id}/asks, maintaining semantic clarity. Every protected endpoint utilizes FastAPI's advanced Dependency Injection system to seamlessly verify JSON Web Tokens (JWT) and inject the current authenticated user context into the route, preventing unauthorized data access.

Data validation is paramount in a peer-to-peer marketplace to prevent database corruption, SQL injection, and malformed content rendering. By defining strict Pydantic schemas, FastAPI automatically validates all incoming JSON request bodies against declared types before any

business logic executes. When a student utilizes the "Ask Mode" to create a request, the AskCreate Pydantic schema ensures the budget_max is a positive numeric value, the title meets specific length boundaries to prevent interface breakage, and the category_id maps to an existing integer. If validation fails, FastAPI instantly intercepts the request and returns a 422 Unprocessable Entity error containing a detailed JSON response indicating precisely which field failed validation. This mechanism drastically reduces the boilerplate code required for error handling and fortifies the application against bad data.

Identity Verification and Trust Engineering

The primary value proposition of CampusTrade over generic, open-network platforms like Craigslist or Facebook Marketplace is the deliberate elimination of the "Hyperlocal Trust Deficit". In open networks, users face significant risks regarding fraudulent listings, physical safety during meetups, and unverified product claims. CampusTrade mitigates these risks by establishing a closed, high-trust ecosystem where participation mandates strict academic identity verification.

Self-Contained University Email Verification Logic

A core requirement of this mini project is the avoidance of complex, expensive third-party integrations. Therefore, instead of relying on commercial verification APIs like SheerID—which require complex OAuth handshakes and recurring licensing fees—CampusTrade relies on self-contained Python logic to execute verification. The standard method for gating academic communities is restricting registration to users possessing an active .edu (or international academic equivalent) email address. However, simplistic string matching (such as checking if a string ends with .edu) is wholly insufficient, as it is easily bypassed by malformed strings or expired domains.

To ensure robust verification without external dependencies, the FastAPI backend implements a sophisticated, multi-tiered Python verification logic within the verification_service.py module:

The first defense layer utilizes Python's native re module to enforce strict syntactic rules. A comprehensive regular expression pattern ensures the presence of localized characters, a single @ symbol, and a properly formatted Top-Level Domain. This initial filter instantly rejects malformed inputs, spaces, and obvious injection attempts without consuming computational resources.

Because valid syntax does not guarantee a functioning or legitimate inbox, the system subsequently utilizes the dnspython library to execute a Domain Name System (DNS) query against the extracted academic domain. By querying the Mail Exchanger (MX) records, the backend verifies that the university's domain is actively configured to receive emails, blocking users attempting to register with dead or parked .edu domains.

Finally, to confirm possession of the inbox without engaging in complex SMTP server handshakes—which risk triggering spam filters—CampusTrade utilizes Supabase's native authentication email capabilities. The system dispatches a secure, time-sensitive Magic Link to the validated email address. The student must access their institutional inbox and click the cryptographic link to activate their account, definitively proving their active academic status and establishing a chain of trust.

Privacy, Security, and Regulatory Alignment

By centralizing identity around the university email, CampusTrade creates a transparent accountability model. Buyers and sellers operate with the assurance that the counterparty is a verified peer, dramatically reducing the likelihood of predatory behavior.
However, managing student data necessitates adherence to stringent privacy principles. While the Family Educational Rights and Privacy Act (FERPA) primarily governs institutional records, applications handling student data must independently implement rigorous security safeguards to maintain user trust. CampusTrade operates on the principle of data minimization: the platform deliberately avoids extracting or storing unnecessary Personally Identifiable Information (PII) from university directories. The .edu email is utilized exclusively as an authentication vector. Any additional sensitive fields—such as mobile numbers provided voluntarily to coordinate on-campus meetups—are heavily encrypted at rest within the Supabase database, ensuring that an infrastructure compromise does not expose vulnerable student data.

The "Ask Mode": Demand-Driven Marketplace Mechanics

A defining feature requested for this project is the integration of an "Ask Mode," which fundamentally alters the transactional dynamics of the application. Traditional marketplaces are overwhelmingly supply-driven; sellers post items, and buyers must continuously sift through generic search results hoping to find a match. This leads to high search friction and platform fatigue.
The "Ask Mode" reverses this paradigm, allowing CampusTrade to function as a demand-driven coordination engine. If a student urgently requires a specific calculus textbook, a laboratory coat of a certain size, or a mini-fridge before the semester begins, they can publish an "Ask" specifying their exact requirements, maximum budget, and timeline.
This feature requires dedicated API endpoints within the FastAPI application. When an "Ask" is created, the backend indexes it within the public.asks table and makes it immediately available on a dedicated "Campus Requests" feed. Students looking to offload items can browse this feed and instantly fulfill existing demand. This mechanism dramatically reduces the time-to-transaction and fosters a collaborative community atmosphere, as users actively help peers source necessary materials. By implementing this feature, CampusTrade moves beyond a simple classifieds board to become a highly efficient, bidirectional campus resource exchange.

Media Storage and On-the-Fly Image Optimization

A visual marketplace relies heavily on the clarity and loading speed of product images. Uploading raw, uncompressed images from modern mobile devices—which frequently exceed 5 to 10 Megabytes per file—directly to a relational database causes severe performance bottlenecks, exorbitant egress bandwidth consumption, and sluggish user experiences.

Supabase Storage Buckets

To manage media efficiently without complex AWS S3 integrations, CampusTrade utilizes Supabase Storage, an object storage solution designed to operate synchronously with the

PostgreSQL database. Media files are segregated into specialized "Files Buckets," which are protected by the same Row Level Security (RLS) policies governing the database.
The media handling workflow is highly decoupled from the database transaction to ensure system resilience. When a student creates a listing or an "Ask" requiring an image, the frontend application transmits the file to the FastAPI backend. The Python service validates the file, ensuring the payload conforms to safe MIME types (e.g., image/jpeg, image/png) and respects predefined size thresholds. The backend then utilizes the supabase-py client library (supabase.storage.from_('media').upload()) to securely push the file into the designated bucket. Upon successful upload, Supabase Storage returns a unique, permanent path for the asset, which FastAPI then inserts into the public.media table, associating the image cryptographically with the corresponding marketplace entity.

Dynamic Image Transformation

To guarantee the marketplace interface remains highly responsive across various devices, CampusTrade leverages Supabase's built-in image optimization and transformation capabilities. Rather than the backend generating and storing multiple thumbnail sizes of a single image—which consumes excess storage and compute—the frontend application requests transformations dynamically via the URL.
When retrieving image URLs, transformation parameters are appended to resize, crop, and compress the asset on the fly. Crucially, the Supabase Storage Content Delivery Network (CDN) automatically evaluates the requesting client's browser capabilities and serves the image in the highly optimized WebP format wherever supported. This dynamic processing radically lowers bandwidth consumption, accelerates page load times, and significantly improves the browsing experience for students relying on varied campus Wi-Fi or cellular networks, all without requiring complex, third-party image processing integrations.

Deployment Strategy and Serverless Infrastructure

Deploying and maintaining web applications traditionally involves provisioning virtual machines, configuring load balancers, and managing ongoing operating system security patches. For a streamlined mini project, this operational overhead detracts from core feature development. Therefore, CampusTrade's backend architecture is engineered for deployment entirely on Vercel's serverless infrastructure.

Vercel Serverless Functions

Vercel provides a highly optimized deployment environment that natively supports Python runtimes, allowing FastAPI applications to be deployed as scalable Serverless Functions. When deployed serverlessly, the FastAPI application does not run continuously. Instead, it scales automatically from zero to thousands of concurrent executions based purely on incoming HTTP traffic.
This "scale-to-zero" capability ensures extreme cost-efficiency, as the project only incurs compute costs when API endpoints are actively evaluating requests. Vercel automatically provisions and renews SSL/HTTPS certificates, guaranteeing encrypted data transit between the student's device and the API without manual intervention or complex Let's Encrypt

configurations.

Configuration and Continuous Integration

Deploying the FastAPI application to Vercel requires precise configuration files at the root of the repository to instruct the platform's build engine.
The primary orchestration file is vercel.json, which defines the build process and routing logic. The configuration directs Vercel to utilize the @vercel/python builder on the main application entry point (e.g., app/main.py) and routes all incoming wildcard traffic (/(.*)) to that specific destination, allowing FastAPI's internal routing matrix to handle the specific endpoint resolution.
To mitigate "cold start" latency—a common issue in serverless deployments—the configuration utilizes excludeFiles directives to strip unnecessary assets (like tests, documentation, and virtual environments) from the final deployment package, ensuring the function remains highly performant.
Concurrently, Vercel automatically parses the requirements.txt file during the build phase to install necessary Python dependencies (such as fastapi, uvicorn, pydantic, and supabase). Pinning exact version numbers in this file is a mandatory best practice to prevent unexpected pipeline failures caused by unvetted upstream library updates.
By integrating Vercel directly with the project's Git repository, a robust Continuous Integration and Continuous Deployment (CI/CD) pipeline is established natively. Every code push to the main branch automatically triggers a new build, executes validation tests, and deploys the updated codebase to a globally distributed Edge Network, ensuring seamless, zero-downtime iterations.

Project Execution and Lifecycle Methodology

The development of a full-stack application, even a scoped mini project like CampusTrade, requires strict adherence to a structured Software Development Life Cycle (SDLC) methodology. Abandoning rigid, linear Waterfall models that resist change, CampusTrade will be executed utilizing an iterative, Agile-based approach. This allows for rapid prototyping, continuous integration of user feedback, and flexible adaptation to emerging technical constraints.
The execution strategy is divided into several distinct, iterative phases to ensure systematic progress:

Execution Phase	Strategic Focus	Deliverables and Milestones
Phase 1: Requirements Analysis	Defining the precise scope and domain modeling of the marketplace.	Entity-Relationship Diagrams (ERD), user persona definitions, and core requirement mapping.
Phase 2: Infrastructure Initialization	Establishing the foundational database and deployment pipelines.	Supabase project creation, declarative SQL schema execution, RLS policy configuration, and Vercel CI/CD setup.
Phase 3: Core Authentication	Developing the backend logic for identity and trust engineering.	Python regex and DNS MX record validation for .edu emails, Supabase Auth integration, and User Profile

Execution Phase	Strategic Focus	Deliverables and Milestones
		endpoints.
Phase 4: Marketplace Mechanics	Engineering the transactional core of the application, including the unique "Ask Mode".	Implementation of supply listings, demand-driven "Asks", category indexing, and Supabase Storage integration for media.
Phase 5: Client Integration	Developing the user interface and connecting it to the backend architecture.	Frontend component construction, API consumption, and state management for seamless user experiences.
Phase 6: Quality Assurance	Executing comprehensive testing across the application pyramid to ensure stability.	Unit testing for business logic, integration tests for database operations, and endpoint payload validation.

Following these phases, the project will culminate in a Beta Launch pushed to the Vercel production environment. This soft launch, targeted at a controlled subset of students, will be utilized to monitor database query performance, identify user experience friction points, and gather early engagement metrics before expanding the platform across the broader campus community.

Conclusion and Future Trajectory

The architectural framework detailed in this project plan provides a highly robust, secure, and performant foundation for the CampusTrade application. By intentionally circumventing complex third-party integrations and instead combining the rapid development capabilities of FastAPI, the relational stability and built-in features of Supabase, and the seamless scalability of Vercel's serverless infrastructure, the engineering team is positioned to deliver a premium software product uniquely suited to the constraints of a mini project.

Crucially, the decision to enforce institutional email verification natively via Python, combined with the innovative "Ask Mode," directly addresses the primary vulnerabilities of localized marketplaces. This strategy transforms a disparate group of buyers and sellers into a high-trust, authenticated academic community capable of bidirectional exchange.

As the platform matures and user adoption increases, the decoupled, domain-driven architecture allows for seamless horizontal scaling without fundamental structural rewrites. Future iterations of CampusTrade can leverage this solid foundation to introduce sophisticated features such as machine learning-driven product recommendations, predictive market pricing algorithms, and real-time WebSocket messaging. By strictly adhering to modern API design principles, comprehensive database normalization, and agile execution methodologies, CampusTrade is engineered not just to launch successfully, but to sustainably resolve the hyperlocal trust deficit inherent in university commerce.

Works cited

1. Connect Sphere: A Cost-Free, Peer-to-Peer Student Marketplace for Hyperlocal University Communities - IJFMR, https://www.ijfmr.com/papers/2025/5/58019.pdf 2. Campus Marketplace App: Building a Peer Selling Platform - EvePaper,

https://evepaper.com/campus-marketplace-app-peer-selling-platform/ 3. Northeastern graduate creates safer student-only marketplace app after being scammed, https://news.northeastern.edu/2025/05/22/iwantit-marketplace-app-college-students/ 4. A college app for resources, anonymous feedback and a student "marketplace" - Reddit, https://www.reddit.com/r/Btechtards/comments/1ozdxrf/a_college_app_for_resources_anonymous_feedback/ 5. How to Build a Professional Marketplace Website in 2026? - Emergent, https://emergent.sh/learn/how-to-build-a-marketplace-website 6. Six ways to build a marketplace platform - Sharetribe, https://www.sharetribe.com/how-to-build/ways-to-build-marketplace-platform/ 7. Building a CRUD API with FastAPI and Supabase: A Step-by-Step Guide - Keshav Malik, https://blog.theinfosecguy.xyz/building-a-crud-api-with-fastapi-and-supabase-a-step-by-step-guide 8. Guide to Building a Marketplace App - Zetaton, https://www.zetaton.com/blogs/a-comprehensive-guide-to-building-a-marketplace-app 9. Best Practices for Structuring a Next.js + FastAPI + Supabase Project? - Discussions, https://forum.cursor.com/t/best-practices-for-structuring-a-next-js-fastapi-supabase-project/49706 10. Building Scalable Applications: A Guide to CRUD APIs with FastAPI and Supabase | Glasp, https://glasp.co/hatch/dorland/p/rcGYZf4iQZAGP93IMw6Y 11. Full Stack FastAPI Template, https://fastapi.tiangolo.com/project-generation/ 12. How to Deploy FastAPI to Vercel - Cleverzone - Medium, https://cleverzone.medium.com/fastapi-deployment-into-vercel-0fa4e6478014 13. Vercel Marketplace | Supabase Docs, https://supabase.com/docs/guides/integrations/vercel-marketplace 14. Structuring a FastAPI Project: Best Practices - DEV Community, https://dev.to/mohammad222pr/structuring-a-fastapi-project-best-practices-53l6 15. FastAPI Best Practices and Conventions we used at our startup - GitHub, https://github.com/zhanymkanov/fastapi-best-practices 16. How to Create a Marketplace App: Key Features for Success in Development, https://wtt-solutions.com/blog/marketplace-app-development-key-features-for-success 17. How to Create a Marketplace App in 7 Steps in 2025 - KITRUM, https://kitrum.com/blog/how-to-create-a-marketplace-app-in-7-steps/ 18. mjhea0/awesome-fastapi - GitHub, https://github.com/mjhea0/awesome-fastapi 19. FastAPI Best Practices: A Complete Guide for Building Production-Ready APIs - Medium, https://medium.com/@abipoongodi1211/fastapi-best-practices-a-complete-guide-for-building-production-ready-apis-bb27062d7617 20. FastAPI Best Practices - Auth0, https://auth0.com/blog/fastapi-best-practices/ 21. Building a Supabase and FastAPI Project: A Modern Backend Stack | by Abhishek Kumar, https://medium.com/@abhik12295/building-a-supabase-and-fastapi-project-a-modern-backend-stack-52030ca54ddf 22. FastAPI, https://fastapi.tiangolo.com/ 23. First Steps - FastAPI, https://fastapi.tiangolo.com/tutorial/first-steps/ 24. Database | Supabase Docs, https://supabase.com/docs/guides/database/overview 25. Supabase | The Postgres Development Platform., https://supabase.com/ 26. Storage | Store any digital content - Supabase, https://supabase.com/storage 27. What is a relational database? - Microsoft Azure, https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-a-relational-database 28. Postgres database | Supabase Features, https://supabase.com/features/postgres-database 29. Storage | Supabase Docs, https://supabase.com/docs/guides/storage 30. Easy functions and transactions using Postgres + PostgREST or Supabase, https://dev.to/voboda/gotcha-supabase-postgrest-rpc-with-transactions-45a7 31. Cal.com

launches Expert Marketplace built with Next.js and Supabase., https://supabase.com/blog/calcom-platform-starter-kit-nextjs-supabase 32. Deploying FastAPI app on Vercel Serverless - DEV Community, https://dev.to/abdadeel/deploying-fastapi-app-on-vercel-serverless-18b1 33. Deploy Your FastAPI App on Vercel: The Complete Guide - DEV Community, https://dev.to/highflyer910/deploy-your-fastapi-app-on-vercel-the-complete-guide-27c0 34. Student Management System Database: Architecture & Selection Guide - OpenEduCat, https://openeducat.org/articles/student-management-system-database-guide/ 35. Declarative database schemas | Supabase Docs, https://supabase.com/docs/guides/local-development/declarative-database-schemas 36. Declarative Schemas | Supabase Features, https://supabase.com/features/declarative-schemas 37. Declarative Schemas for Simpler Database Management - Supabase, https://supabase.com/blog/declarative-schemas 38. Tables and Data | Supabase Docs, https://supabase.com/docs/guides/database/tables 39. User Management | Supabase Docs, https://supabase.com/docs/guides/auth/managing-user-data 40. Bigger Applications - Multiple Files - FastAPI, https://fastapi.tiangolo.com/tutorial/bigger-applications/ 41. FastAPI Project Structure: Production Architecture Guide (2026) - Zestminds, https://www.zestminds.com/blog/fastapi-project-structure/ 42. Production-Ready FastAPI Project Structure (2026 Guide) - DEV Community, https://dev.to/thesius_code_7a136ae718b7/production-ready-fastapi-project-structure-2026-guide-b1g 43. Building a CRUD FastAPI app with SQLAlchemy - Mattermost, https://mattermost.com/blog/building-a-crud-fastapi-app-with-sqlalchemy/ 44. 15 FastAPI Project Ideas For Data Scientists - ProjectPro, https://www.projectpro.io/article/fastapi-projects/847 45. SQL (Relational) Databases - FastAPI, https://fastapi.tiangolo.com/tutorial/sql-databases/ 46. Validating User Input with FastAPI: An Example with Custom Validators - DEV Community, https://dev.to/blamsa0mine/-validating-user-input-with-fastapi-an-example-with-custom-validators-24l0 47. Full stack, modern web application template. Using FastAPI, React, SQLModel, PostgreSQL, Docker, GitHub Actions, automatic HTTPS and more., https://github.com/fastapi/full-stack-fastapi-template 48. Providing a Secure Digital Marketplace for College Students to Participate in E-Commerce Activity - libra etd, https://libraetd.lib.virginia.edu/public_view/8336h302c 49. UniExchange - Campus Marketplace, https://uniexchange.app/ 50. Campus Commerce: Peer-to-Peer Student Marketplace | StartupLab - Startup.ai, https://startup.ai/startup-idea/campus-commerce-peer-to-peer-student-marketplace-5aabb8f5 51. SheerID: Identity Marketing Platform, https://www.sheerid.com/ 52. Student Verifications outside USA and organizations without .edu email - Feature Requests, https://forum.cursor.com/t/student-verifications-outside-usa-and-organizations-without-edu-email/133736 53. Python Email Verification Guide | ZeroBounce, https://www.zerobounce.net/email-guides/python-email-verification 54. How to verify a .edu using a python script? - Stack Overflow, https://stackoverflow.com/questions/28007924/how-to-verify-a-edu-using-a-python-script 55. Email Validation in Python: Complete Guide | by ryan - Medium, https://medium.com/@ryan_forrester_/email-validation-in-python-complete-guide-3b9a5f6c880a 56. 3 Methods for Email Validation Using Python (Tutorial and Code) - MailerCheck, https://www.mailercheck.com/articles/email-validation-using-python 57. Python Email Validation: Tutorial with Code Snippets [2026] - Mailtrap, https://mailtrap.io/blog/python-validate-email/ 58. Email Validation in Python – Step-by-Step Guide to Validate Email Addresses! - coding courses, https://mahaviracodingclasses.graphy.com/blog/email-validation-in-python 59. Automating Email

Domain Verification with Python | by Mathi Maheswaran - Medium, https://mathimaheswaran.medium.com/automating-email-domain-verification-with-python-2c514987abf7 60. python - What is the best way to verify an email address if it actually exist? - Stack Overflow, https://stackoverflow.com/questions/69412522/what-is-the-best-way-to-verify-an-email-address-if-it-actually-exist 61. Rumie - College Marketplace App Project by CrestCoder, Educational Connection Platform, https://www.crestcoder.com/projects/rumie-app 62. Applying Google Sheets and FastAPI for Email Verification | by Denis Bélanger - Medium, https://medium.com/@python-javascript-php-html-css/applying-google-sheets-and-fastapi-for-email-verification-ac462d51308c 63. Student Identity Verification in Online Learning - Fashion Institute of Technology, https://www.fitnyc.edu/academics/online-learning/identity-verification.php 64. Identity Authentication Best Practices - Protecting Student Privacy - Department of Education, https://studentprivacy.ed.gov/resources/identity-authentication-best-practices 65. FERPA Identity Verification | GW Privacy Office | The George Washington University, https://privacy.gwu.edu/ferpa-identity-verification 66. Student Identity Verification for Enrollment through Third-Party Learning Platforms, https://www.colorado.edu/compliance/policies/student-identity-verification-enrollment-through-third-party-learning-platforms 67. Why email is a poor way to identify students - InAcademia, https://inacademia.org/email-validation/ 68. pakeezaanjum/campus-marketplace - GitHub, https://github.com/pakeezaanjum/campus-marketplace 69. 10 Tips For Students To Protect Data Privacy | archTIS - Spirion, https://www.spirion.com/blog/10-data-privacy-tips-for-staff-and-students-in-education 70. Introduction - CampusKart - Mintlify, https://www.mintlify.com/rawatpriyansh17/DBMS-Project-CampusKart/introduction 71. Storage Image Transformations | Supabase Docs, https://supabase.com/docs/guides/storage/serving/image-transformations 72. Best Practices for Your Digital Campus App Design - Modo Labs, https://modolabs.com/type/blog/best-practices-for-your-digital-campus-app-design/ 73. Best Practices for using images from mobile devices : r/Supabase - Reddit, https://www.reddit.com/r/Supabase/comments/1c1376x/best_practices_for_using_images_from_mobile/ 74. Storage Quickstart | Supabase Docs, https://supabase.com/docs/guides/storage/quickstart 75. Python: Upload a file | Supabase Docs, https://supabase.com/docs/reference/python/storage-from-upload 76. Handling image storage with Supabase, on the Frontend | by Gbadamosi Mateen | Medium, https://medium.com/@gbadamosimateen/handling-image-storage-with-supabase-on-the-frontend-da328717121f 77. How do i upload an image to supabase and have the image preview in the UI? - Reddit, https://www.reddit.com/r/Supabase/comments/18q5lus/how_do_i_upload_an_image_to_supabase_and_have_the/ 78. Build a Marketplace App: Features, Costs, and Step-by-Step Process - Journeyhorizon, https://www.journeyh.io/blog/build-a-marketplace-app 79. Deploy a FastAPI app on Vercel, https://vercel.com/docs/frameworks/backend/fastapi 80. Deploy FastAPI app on Vercel Serverless | by abdadeel - Medium, https://abdadeel.medium.com/deploy-fastapi-app-on-vercel-serverless-b9fc35bba74d 81. Building and Deploying My First FastAPI Application on Vercel: A Complete Journey | by Masud Pervez | Medium, https://medium.com/@masud.pervez27/building-and-deploying-my-first-fastapi-application-on-vercel-a-complete-journey-7b010345162c 82. 6 Project Management Methodologies Explained - Everglades University,

https://www.evergladesuniversity.edu/blog/project-management-methodologies/ 83. A Comprehensive Review of Software Development Life Cycle methodologies: Pros, Cons, and Future Directions, https://ijcsm.researchcommons.org/cgi/viewcontent.cgi?article=1114&context=ijcsm 84. (PDF) Software Development Life Cycle (SDLC) Methodologies for Information Systems Project Management - ResearchGate, https://www.researchgate.net/publication/373800862_Software_Development_Life_Cycle_SDLC_Methodologies_for_Information_Systems_Project_Management